



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра математического обеспечения и стандартизации информационных технологий(МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Распределенные системы управления базами данных»

Практическое занятие № 4

Студент группы

ИКБО-65-23, Олефиоров Георгий Гурамович

(подпись)

Преподаватель

Фандеев Илья Игоревич

(подпись)

Отчет представлен

«__»_____202__г.

Москва 2026 г.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Особенности при создании таблиц Cassandra

ОТСУТСТВИЕ JOIN

В Cassandra нет опции join. Чаще всего, это ограничение обходится с помощью денормализации данных по дополнительным таблицам.

ОТСУТСТВИЕ ВНЕШНИХ КЛЮЧЕЙ

В Cassandra нет способа связывать сущности из разных таблиц между собой, как например, это сделано в реляционных базах с помощью внешних ключей. Как следствие отсутствуют такие операции как каскадное удаление. ДЕНОРМАЛИЗАЦИЯ

Сама по себе денормализация обладает одним достоинством, в отличие от обычного подход. Ее удобно использовать для хранения исторических данных, которые ни при каких обстоятельствах не должны изменяться. В качестве примера можно привести хранение покупок клиента со ссылками на товар. Т.к. цена товара меняется с течением времени, чтобы получить стоимость каждого товара на момент приобретения, нужно отдельно хранить цену товара. При использовании подхода с денормализацией, можно в таблице с покупками хранить полностью объект товара. При этом текущий товар можно изменять любым образом и даже удалять и это никак не повлияет на историю покупок клиента.

ЗАПРОС ПЕРВИЧЕН

При проектировании, нужно учитывать, что запрос должен получить за один раз все данные из одной таблицы (т.к. нет join). Поэтому сначала проектируются все возможные запросы, а затем под них создаются таблицы. Таким образом в Cassandra создание структуры БД начинается с определения запросов.

ОПТИМАЛЬНОЕ ХРАНЕНИЕ

В реляционных базах редко можно встретить рекомендации к структуре БД для оптимального хранения и чтения данных. Чаще всего никто не заботится об этом. В распределенных системах дело обстоит иначе, т.к. данные располагаются на нескольких узлах, более высокую производительность будет демонстрировать тот запрос, который отдает данные с одной ноды. Таким образом, желательно располагать данные так, чтобы данные возвращались из одной ноды.

Практическая часть:

Листинг 1 – *docker-compose.yml*

```
services:
  cassandra-seed-olefirov:
    image: cassandra:4.1
    container_name: cassandra-seed-olefirov
    environment:
      - CASSANDRA_CLUSTER_NAME=OlefirovGG
      - CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch
      - CASSANDRA_DC=DC_Moscow
      - AUTO_BOOTSTRAP=false
    ports:
      - "19042:9042"
    volumes:
      - cassandra_data_seed:/var/lib/cassandra

  cassandra-node-olefirov_1:
    image: cassandra:4.1
    container_name: cassandra-node-olefirov_1
    environment:
      - CASSANDRA_CLUSTER_NAME=OlefirovGG
      - CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch
      - CASSANDRA_DC=DC_Moscow
      - CASSANDRA_SEEDS=cassandra-seed-olefirov
    ports:
      - "19043:9042"
    volumes:
      - cassandra_data_node_1:/var/lib/cassandra
    depends_on:
      - cassandra-seed-olefirov

  cassandra-node-olefirov_2:
    image: cassandra:4.1
    container_name: cassandra-node-olefirov_2
    environment:
      - CASSANDRA_CLUSTER_NAME=OlefirovGG
      - CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch
      - CASSANDRA_DC=DC_Spb
      - CASSANDRA_SEEDS=cassandra-seed-olefirov
    ports:
      - "19044:9042"
    volumes:
      - cassandra_data_node_2:/var/lib/cassandra
    depends_on:
      - cassandra-seed-olefirov

volumes:
  cassandra_data_seed:
  cassandra_data_node_1:
  cassandra_data_node_2:
```

cassandra_data_node_2:

```
C:\Users\Gorgi\Downloads>docker compose up -d
time="2026-04-06T21:41:47+03:00" level=warning msg="C:\\Users\\Gorgi\\Downloads\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 18/19
✓cassandra-node-olefirov_2 Pulled          72.7s
✓cassandra-seed-olefirov Pulled           73.0s
✓cassandra-node-olefirov_1 Pulled         72.7s
✓b2ac7efcaf8 Pull complete               3.6s
✓bd87013b97f2 Pull complete              36.7s
✓2c18c4e31ee3 Pull complete              66.0s
✓af3ff48d1b4f Pull complete              66.1s
✓67f678c91cdb Pull complete              11.1s
✓4b5995275380 Pull complete              36.2s
✓031d970efc58 Pull complete              66.9s
[+] Running 6/6
✓Volume "downloads_cassandra_data_olefirov_node2" Created      0.0s
✓Volume "downloads_cassandra_data_olefirov_seed" Created       0.0s
✓Volume "downloads_cassandra_data_olefirov_node1" Created       0.0s
✓Container cassandra-seed-olefirov Started                     3.6s
✓Container cassandra-node-olefirov_2 Started                   1.5s
✓Container cassandra-node-olefirov_1 Started                   1.6s
C:\Users\Gorgi\Downloads>
```

Рисунок 1 – Запуск docker compose для создания кластера

```
C:\aynik>docker exec -it cassandra-seed-olefirov cqlsh

ATTENTION: All commands will be saved to history file: /root/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history] section of cqlshrc.
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to OlefirovGG at 127.0.0.1:9042
[cqlsh 6.1.0 | Cassandra 4.1.11 | CQL spec 3.4.6 | Native protocol v5]
Use HELP for help.
cqlsh> DROP KEYSPACE IF EXISTS learn_cassandra;
cqlsh> CREATE KEYSPACE learn_cassandra
... WITH REPLICATION = {
...   'class' : 'NetworkTopologyStrategy',
...   'DC_Moscow' : 2
... };
cqlsh>
```

Рисунок 2 – Подключение к контейнеру и создание пространства ключей

```
cqlsh:learn_cassandra> CREATE TABLE users_by_country (
...   country text,
...   user_email text,
...   first_name text,
...   last_name text,
...   age smallint,
...   PRIMARY KEY ((country), user_email)
... );
cqlsh:learn_cassandra> INSERT INTO users_by_country (country, user_email, first_name, last_name, age)
... VALUES ('US', 'john@email.com', 'John', 'Wick', 55);
cqlsh:learn_cassandra> INSERT INTO users_by_country (country, user_email, first_name, last_name, age)
... VALUES ('UK', 'peter@email.com', 'Peter', 'Clark', 65);
cqlsh:learn_cassandra> INSERT INTO users_by_country (country, user_email, first_name, last_name, age)
... VALUES ('UK', 'bob@email.com', 'Bob', 'Sandler', 23);
cqlsh:learn_cassandra> INSERT INTO users_by_country (country, user_email, first_name, last_name, age)
... VALUES ('UK', 'alice@email.com', 'Alice', 'Brown', 26);
cqlsh:learn_cassandra> SELECT * FROM users_by_country;

country | user_email | age | first_name | last_name
-----+-----+-----+-----+-----
US | john@email.com | 55 | John | Wick
UK | alice@email.com | 26 | Alice | Brown
UK | bob@email.com | 23 | Bob | Sandler
UK | peter@email.com | 65 | Peter | Clark

(4 rows)
```

Рисунок 3 – Создание таблицы users_by_country с последующим её заполнением

```

cqlsh:learn_cassandra> CREATE TABLE users_by_email (
    ...     user_email text,
    ...     country text,
    ...     first_name text,
    ...     last_name text,
    ...     age smallint,
    ...     PRIMARY KEY (user_email)
    ... );
cqlsh:learn_cassandra>
cqlsh:learn_cassandra> INSERT INTO users_by_email (user_email, country, first_name, last_name, age)
    ... VALUES ('john@email.com', 'US', 'John', 'Wick', 55);
cqlsh:learn_cassandra>
cqlsh:learn_cassandra> INSERT INTO users_by_email (user_email, country, first_name, last_name, age)
    ... VALUES ('peter@email.com', 'UK', 'Peter', 'Clark', 65);
cqlsh:learn_cassandra>
cqlsh:learn_cassandra> INSERT INTO users_by_email (user_email, country, first_name, last_name, age)
    ... VALUES ('bob@email.com', 'UK', 'Bob', 'Sandler', 23);
cqlsh:learn_cassandra>
cqlsh:learn_cassandra> INSERT INTO users_by_email (user_email, country, first_name, last_name, age)
    ... VALUES ('alice@email.com', 'UK', 'Alice', 'Brown', 26);
cqlsh:learn_cassandra>
cqlsh:learn_cassandra> SELECT * FROM users_by_email;

user_email | age | country | first_name | last_name
-----+-----+-----+-----+-----
john@email.com | 55 | US | John | Wick
alice@email.com | 26 | UK | Alice | Brown
peter@email.com | 65 | UK | Peter | Clark
bob@email.com | 23 | UK | Bob | Sandler

(4 rows)

```

Рисунок 4 – Создание таблицы users_by_email с последующим её заполнением

```

(4 rows)
cqlsh:learn_cassandra> SELECT * FROM users_by_email WHERE age = 26 ALLOW FILTERING;

user_email | age | country | first_name | last_name
-----+-----+-----+-----+-----
alice@email.com | 26 | UK | Alice | Brown

(1 rows)
cqlsh:learn_cassandra> SELECT * FROM users_by_email WHERE age = 55 ALLOW FILTERING;

user_email | age | country | first_name | last_name
-----+-----+-----+-----+-----
john@email.com | 55 | US | John | Wick

(1 rows)

```

Рисунок 5 – Фильтрация по столбцу, который не является ключом раздела

```

(1 rows)
cqlsh:learn_cassandra> CONSISTENCY ALL;
Consistency level set to ALL.
cqlsh:learn_cassandra> SELECT * FROM users_by_country WHERE country = 'US';

country | user_email | age | first_name | last_name
-----+-----+-----+-----+-----
US | john@email.com | 55 | John | Wick

(1 rows)
cqlsh:learn_cassandra> |

```

Рисунок 6 – Установление уровня согласованности «ALL» с последующим получением данных

```
cqlsh:learn_cassandra> CONSISTENCY ONE;
Consistency level set to ONE.
cqlsh:learn_cassandra> SELECT * FROM users_by_country WHERE country = 'US';
```

country	user_email	age	first_name	last_name
US	john@email.com	55	John	Wick

```
(1 rows)
cqlsh:learn_cassandra> |
```

Рисунок 7 – Установление уровня согласованности «ONE» с последующим получением данных

```
(1 rows)
cqlsh:learn_cassandra> DESCRIBE TABLE users_by_country;
```

```
CREATE TABLE learn_cassandra.users_by_country (
  country text,
  user_email text,
  age smallint,
  first_name text,
  last_name text,
  PRIMARY KEY ((country), user_email)
) WITH CLUSTERING ORDER BY (user_email ASC)
AND additional_write_policy = '99p'
AND bloom_filter_fp_chance = 0.01
AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
AND cdc = false
AND comment = ''
AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
AND memtable = 'default'
AND crc_check_chance = 1.0
AND default_time_to_live = 0
AND extensions = {}
AND gc_grace_seconds = 864000
AND max_index_interval = 2048
AND memtable_flush_period_in_ms = 0
AND min_index_interval = 128
AND read_repair = 'BLOCKING'
AND speculative_retry = '99p';
cqlsh:learn_cassandra> |
```

Рисунок 8 – Определение стратегий для таблицы users_by_country

```
cqlsh:learn_cassandra> CREATE TABLE users_by_country_with_levelled_compaction (
...   country text,
...   user_email text,
...   first_name text,
...   last_name text,
...   age smallint,
...   PRIMARY KEY ((country), user_email)
... ) WITH compaction = { 'class' : 'LevelledCompactionStrategy' };

cqlsh:learn_cassandra>
cqlsh:learn_cassandra> DESCRIBE TABLE users_by_country_with_levelled_compaction;
```

```
CREATE TABLE learn_cassandra.users_by_country_with_levelled_compaction (
  country text,
  user_email text,
  age smallint,
  first_name text,
  last_name text,
  PRIMARY KEY ((country), user_email)
) WITH CLUSTERING ORDER BY (user_email ASC)
AND additional_write_policy = '99p'
AND bloom_filter_fp_chance = 0.1
AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
AND cdc = false
AND comment = ''
AND compaction = {'class': 'org.apache.cassandra.db.compaction.LevelledCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
AND memtable = 'default'
AND crc_check_chance = 1.0
AND default_time_to_live = 0
AND extensions = {}
AND gc_grace_seconds = 864000
AND max_index_interval = 2048
AND memtable_flush_period_in_ms = 0
AND min_index_interval = 128
AND read_repair = 'BLOCKING'
AND speculative_retry = '99p';
cqlsh:learn_cassandra> |
```

Рисунок 9 – Создание таблицы с указанием стратегии сжатия и проверка результата

```

cqlsh:learn_cassandra>
cqlsh:learn_cassandra> CREATE TABLE users_by_country_sorted_by_age_asc (
...     country text,
...     user_email text,
...     first_name text,
...     last_name text,
...     age smallint,
...     PRIMARY KEY ((country), age, user_email)
... ) WITH CLUSTERING ORDER BY (age ASC);
cqlsh:learn_cassandra>
cqlsh:learn_cassandra> INSERT INTO users_by_country_sorted_by_age_asc (country, user_email, first_name, last_name, age)
... VALUES ('US', 'john@email.com', 'John', 'Wick', 10);
cqlsh:learn_cassandra>
cqlsh:learn_cassandra> INSERT INTO users_by_country_sorted_by_age_asc (country, user_email, first_name, last_name, age)
... VALUES ('UK', 'peter@email.com', 'Peter', 'Clark', 30);
cqlsh:learn_cassandra>
cqlsh:learn_cassandra> INSERT INTO users_by_country_sorted_by_age_asc (country, user_email, first_name, last_name, age)
... VALUES ('UK', 'bob@email.com', 'Bob', 'Sandler', 20);
cqlsh:learn_cassandra>
cqlsh:learn_cassandra> INSERT INTO users_by_country_sorted_by_age_asc (country, user_email, first_name, last_name, age)
... VALUES ('UK', 'alice@email.com', 'Alice', 'Brown', 40);
cqlsh:learn_cassandra>
cqlsh:learn_cassandra> -- Проверка сортировки по возрасту (ASC)
cqlsh:learn_cassandra> SELECT * FROM users_by_country_sorted_by_age_asc;

country | age | user_email | first_name | last_name
-----+---+-----+-----+-----
US      | 10 | john@email.com | John      | Wick
UK      | 20 | bob@email.com  | Bob       | Sandler
UK      | 30 | peter@email.com | Peter     | Clark
UK      | 40 | alice@email.com | Alice     | Brown

(4 rows)
cqlsh:learn_cassandra>

```

Рисунок 10 – Создание таблицы с определением столбца кластеризации для сортировки данных с последующей вставкой информации

Заключение:

В ходе работы были рассмотрены ключевые особенности работы с базой данных Cassandra, включая отсутствие операций JOIN и внешних ключей, необходимость денормализации данных, а также важность проектирования структуры таблиц исходя из запросов. Было отмечено, что оптимальное хранение данных в распределённой среде требует учитывать их расположение на узлах для повышения производительности.

Практическая часть включала в себя создание кластера из трёх узлов с использованием Docker Compose, изучение работы с ключами в Cassandra и анализ подходов к обеспечению согласованности данных. В результате были получены знания и навыки, необходимые для проектирования и развертывания распределённых баз данных на основе Cassandra, что позволит эффективно управлять данными в системах с высокой нагрузкой.

Вопросы для самопроверки:

1. Какой Snitch вы выбрали при создании docker-compose и почему? Какие варианты Snitch для Cassandra бывают?

При настройке кластера Cassandra в Docker был выбран GossipingPropertyFileSnitch, так как он обеспечивает автоматическое распространение информации о топологии через gossip-протокол, поддерживает работу с несколькими датацентрами (datacenter1 и datacenter2) и позволяет гибко настраивать DC/рэки через переменные окружения, что критично для динамических окружений с контейнерами. Этот Snitch минимизирует ручное вмешательство в конфигурацию и соответствует рекомендациям для продакшен-сред.

В Apache Cassandra доступны различные варианты Snitch, включая **SimpleSnitch**, **RackInferringSnitch**, **GossipingPropertyFileSnitch** и другие. **GossipingPropertyFileSnitch** является наиболее рекомендуемым, так как использует информацию о датацентрах и стойках из cassandra-rackdc.properties и учитывает динамические изменения кластера. **SimpleSnitch** подходит для

одноцентровых развертываний, а **RackInferringSnitch** автоматически определяет стойки и датацентры по IP-адресам.

2. Укажите формулу уровней согласованности, которая обеспечивает высокую согласованность данных:

Формула уровней согласованности в Apache Cassandra определяется следующим образом: **$R+W>N$**

где:

- **R** — уровень согласованности для операций чтения (количество узлов, с которых должно быть получено подтверждение чтения).
- **W** — уровень согласованности для операций записи (количество узлов, которые должны подтвердить запись).
- **N** — фактор репликации (количество узлов, хранящих копию данных).

Для обеспечения высокой согласованности данных обычно выбирают кворумную стратегию: **$R=W=N/2+1$** что гарантирует пересечение множеств узлов при операциях чтения и записи, обеспечивая актуальность данных.